

Nama : Kafilatul hima

NIM : 250631100080

Kelas : 2D

PENJELASAN

1. Inheritance

Terjadi pada bagian :

```
class Mobil extends Kendaraan
```

Dan :

```
class Mobil extends Kendaraan
```

Kata Kunci *Extends* menunjukkan bahwa class mobil dan motor mewarisi atribut dan method dari class *kendaraan*.

Atribut yang diwarisi :

```
String merk;  
String warna;
```

Method yang diwarisi :

```
void tampilData()  
void jenis()
```

Karena menggunakan inheritance, class mobil dan motor tidak perlu membuat ulang atribut merk dan warna,

2. Constructor Inheritance

Pada constructor child class terdapat :

```
super(merk, warna);
```

Fungsi `super ()` adalah memanggil constructor dari class induk (Kendaraan) agar atribut merk dan warna dapat diisi.

Contoh :

```
Mobil(String merk, String warna, int jumlahPintu) {  
    super(merk, warna);  
    this.jumlahPintu = jumlahPintu;  
}
```

3.Method Overriding

Method Overriding terjadi Ketika method pada parent class dibuat ulang pada child class.

Contohnya :

```
@Override  
void jenis() {  
    System.out.println("Jenis: Mobil");  
}
```

Dan :

```
@Override  
void jenis() {  
    System.out.println("Jenis: Motor");  
}
```

Method jenis () pada class Kendaraan diganti sesuai kebutuhan masing -masing class turunan.

4.Polymorphism

Polymorphism terjadi pada bagian :

```
Kendaraan k1 = new Mobil("Toyota", "Hitam", 4);  
  
    Kendaraan k2 = new Motor("Honda", "Merah",  
"Matic");
```

Variabel k1 dan k2 bertipe kendaraan,tetapi menyimpan objek mobil dan motor.

Saat dipanggil :

```
k1.tampilData();  
  
    k1.jenis();
```

Java akan menjalankan method milik objek mobil.

Begitu juga :

```
k2.tampilData();  
  
    k2.jenis();
```

Java akan menjalankan method milik objek motor.

Source Code

```
class Kendaraan {  
    String merk;  
    String warna;  
  
    Kendaraan(String merk, String warna) {  
        this.merk = merk;  
        this.warna = warna;  
    }  
    void tampilData() {  
        System.out.println("Merk : " + merk);  
        System.out.println("Warna : " + warna);  
    }  
    void jenis() {  
        System.out.println("Jenis Kendaraan");  
    }  
}  
  
class Mobil extends Kendaraan {  
    int jumlahPintu;  
  
    Mobil(String merk, String warna, int jumlahPintu) {  
        super(merk, warna);  
        this.jumlahPintu = jumlahPintu;  
    }  
    @Override  
    void jenis() {  
        System.out.println("Jenis: Mobil");  
    }  
    @Override  
    void tampilData() {  
        super.tampilData();  
        System.out.println("Jumlah Pintu : " + jumlahPintu);  
    }  
}
```

```

class Motor extends Kendaraan {

    String tipe;

    Motor(String merk, String warna, String tipe) {
        super(merk, warna);
        this.tipe = tipe;
    }

    @Override
    void jenis() {
        System.out.println("Jenis: Motor");
    }

    @Override
    void tampilData() {
        super.tampilData();
        System.out.println("Tipe : " + tipe);
    }
}

public class MainKendaraan {

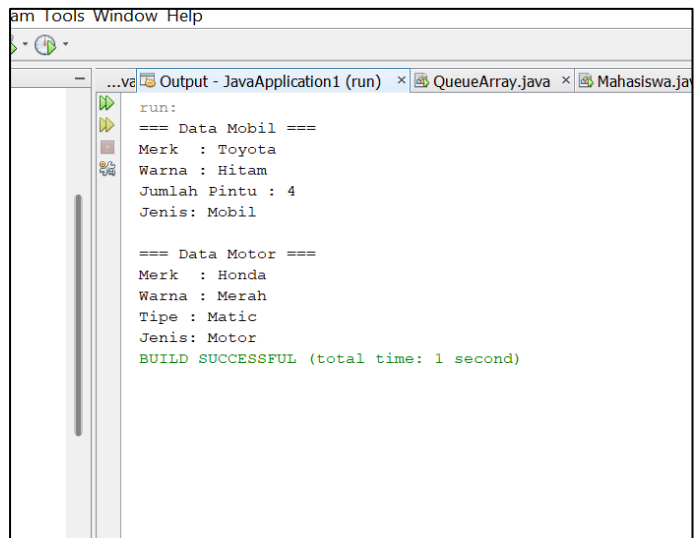
    public static void main(String[] args) {
        // Polymorphism

        Kendaraan k1 = new Mobil("Toyota", "Hitam", 4);
        Kendaraan k2 = new Motor("Honda", "Merah", "Matic");

        System.out.println("=== Data Mobil ===");
        k1.tampilData();
        k1.jenis();
        System.out.println();
        System.out.println("=== Data Motor ===");
        k2.tampilData();
        k2.jenis();
    }
}

```

Input



The screenshot shows an IDE's output window with the following content:

```
run:
=== Data Mobil ===
Merk : Toyota
Warna : Hitam
Jumlah Pintu : 4
Jenis: Mobil

=== Data Motor ===
Merk : Honda
Warna : Merah
Tipe : Matic
Jenis: Motor
BUILD SUCCESSFUL (total time: 1 second)
```

The window title bar includes "am Tools Window Help" and the tab bar shows "Output - JavaApplication1 (run)", "QueueArray.java", and "Mahasiswa.java".